

## 春季class4

题目主要是打暴力。(后面因为纯纯的dfs暴力实在不好找又加了一些分块暴力)

有的题目是本来就是暴力题，有的则是可以暴力拿小评测用例的部分分（这些题我们今天只管部分分，所以不用在意题目的颜色）。

本讲义代码中，可能含有笔者的一些习惯性的 `define`，大致如下：

```
1 using ll = long long;
2 using ull = unsigned long long;
3 using lll = __int128;
4 using pll = pair<ll, ll>;
5 #define fp(i, l, r) for(int i=(l); i<=(r); ++i)
6 #define fq(i, r, l) for(int i=(r); i>=(l); --i)
7 #define fi first
8 #define se second
```

### 第一题： [蓝桥杯 2025 省 A 扫地机器人](#)

去年蓝桥杯的压轴题，我们只拿部分分就跑路。

看一眼小评测用例的范围。

对于 20% 的评测用例， $1 \leq n \leq 5000$ ；

那还说什么了，只要搞个复杂度  $O(n^2)$  的做法，部分分不就到手了吗？

第一想法，肯定是考虑对每个点做起点，都跑一遍 *dfs*，然后答案取最大值。

对这个做法进行一个复杂度的分析。因为图中有  $n$  个点  $n$  条边且保证联通，所以图中只有一个环。

那么在每个 *dfs* 的过程中，每条边最多经过两次，最多经过  $2n$  条边，复杂度  $O(n)$ ，可行。

要注意一个点，主播第一次写的时候惯性思维给点标 *vis* 了然后 *WA*，但实际上这题点是可以重复走的，是边只能走一次，所以要给边开一个 *vis*。

然而，主播改完依旧 *WA* 了，因为改成了只对边开 *vis*，重复走的点会重复贡献。

```
1 int n;
2 int ans[N];
3 int t[N];
4 vector<pair<int,int>> G[N];
5 bool vis1[N]; // 边的vis
6 int vis2[N]; // 点的vis
7
8 int dfs(int u){
9     int maxn=0;
10    vis2[u]++;
11    fp(i,0,G[u].size()-1){
12        int v=G[u][i].fi;
13        int id=G[u][i].se;
14        if(vis1[id]) continue;
15        vis1[id]=1;
16        maxn=max(maxn,dfs(v));
17        vis1[id]=0;
```

```

18     }
19     vis2[u]--;
20     return (vis2[u] ? 0:t[u])+maxn;
21 }
22
23 void solve(){
24     cin>>n;
25     fp(i,1,n) cin>>t[i];
26     fp(i,1,n){
27         int u,v;
28         cin>>u>>v;
29         G[u].push_back({v,i});
30         G[v].push_back({u,i});
31     }
32     fp(i,1,n) ans[i]=dfs(i);
33     int res=0;
34     fp(i,1,n) res=max(res,ans[i]);
35     cout<<res;
36 }

```

|                            |                            |                             |                            |                            |                            |                            |
|----------------------------|----------------------------|-----------------------------|----------------------------|----------------------------|----------------------------|----------------------------|
| #1<br>AC<br>989ms/16.24MB  | #2<br>AC<br>935ms/16.25MB  | #3<br>TLE<br>1.20s/51.00MB  | #4<br>TLE<br>1.20s/36.61MB | #5<br>TLE<br>1.20s/36.25MB | #6<br>TLE<br>1.20s/36.79MB | #7<br>TLE<br>1.20s/51.75MB |
| #8<br>TLE<br>1.20s/52.24MB | #9<br>TLE<br>1.20s/50.25MB | #10<br>TLE<br>1.20s/52.50MB |                            |                            |                            |                            |

可以看到，跑出来的时间是很紧的，给了1s，跑了990ms。所以打完暴力没事干可以想办法优化一下常数。

## 第二题： [蓝桥杯 2025 省 B 第二场 翻转硬币](#)

依旧拿部分分跑路（

**对于 30% 的评测用例， $n, m \leq 20$ ;**

如果直接枚举每一行的翻转情况，然后对每个点进行计算的话，复杂度会到达 $O(2^n * nm)$ ，很难不T。

这时候我们可以先预处理最开始的答案，然后每次翻转更新答案，这样的复杂度是 $O(2^n * m)$ ，完工。

```

1  ll ans=0;
2  ll maxn=0;
3  int n,m;
4  string s[N];
5  int a[N][N];
6  int cnt[N][N];
7
8  void dfs(int dep){
9      if(dep==n+1) {
10         maxn=max(maxn,ans);
11         return;

```

```

12     }
13
14     dfs(dep+1); // 直接不翻这一行
15
16     // 翻这一行的更新
17     ll preans=ans;
18     fp(i,1,m){
19         int delta=0; // 对当前格子的影响
20         if(dep!=1){
21             if(a[dep][i]!=a[dep-1][i]) {
22                 delta++;
23                 ans+=(cnt[dep-1][i]+1)*(cnt[dep-1][i]+1)-cnt[dep-1]
[i]*cnt[dep-1][i];
24                 cnt[dep-1][i]++;
25             }
26             else{
27                 delta--;
28                 ans+=(cnt[dep-1][i]-1)*(cnt[dep-1][i]-1)-cnt[dep-1]
[i]*cnt[dep-1][i];
29                 cnt[dep-1][i]--;
30             }
31         }
32         if(dep!=n){
33             if(a[dep][i]!=a[dep+1][i]) {
34                 delta++;
35                 ans+=(cnt[dep+1][i]+1)*(cnt[dep+1][i]+1)-cnt[dep+1]
[i]*cnt[dep+1][i];
36                 cnt[dep+1][i]++;
37             }
38             else{
39                 delta--;
40                 ans+=(cnt[dep+1][i]-1)*(cnt[dep+1][i]-1)-cnt[dep+1]
[i]*cnt[dep+1][i];
41                 cnt[dep+1][i]--;
42             }
43         }
44         ans+=(cnt[dep][i]+delta)*(cnt[dep][i]+delta)-cnt[dep][i]*cnt[dep]
[i];
45         cnt[dep][i]+=delta;
46     }
47     fp(i,1,m) a[dep][i]=1-a[dep][i];
48     dfs(dep+1);
49     //回退
50     ans=preans;
51     fp(i,1,m){
52         int delta=0;
53         if(dep!=1){
54             if(a[dep][i]!=a[dep-1][i]) {
55                 delta++;
56                 cnt[dep-1][i]++;
57             }
58             else{
59                 delta--;
60                 cnt[dep-1][i]--;
61             }
62         }

```

```

63         if(dep!=n){
64             if(a[dep][i]!=a[dep+1][i]) {
65                 delta++;
66                 cnt[dep+1][i]++;
67             }
68             else{
69                 delta--;
70                 cnt[dep+1][i]--;
71             }
72         }
73         cnt[dep][i]+=delta;
74     }
75     fp(i,1,m) a[dep][i]=1-a[dep][i];
76 }
77
78 void solve(){
79     cin>>n>>m;
80     fp(i,1,n) cin>>s[i];
81     fp(i,1,n) fp(j,0,m-1) {
82         if(s[i][j]=='0') a[i][j+1]=0;
83         else a[i][j+1]=1;
84     }
85     fp(i,1,n) fp(j,1,m){
86         if(i!=1 and a[i-1][j]==a[i][j]) cnt[i][j]++;
87         if(i!=n and a[i+1][j]==a[i][j]) cnt[i][j]++;
88         if(j!=1 and a[i][j-1]==a[i][j]) cnt[i][j]++;
89         if(j!=m and a[i][j+1]==a[i][j]) cnt[i][j]++;
90         ans+=cnt[i][j]*cnt[i][j];
91     }
92     dfs(1);
93     cout<<maxn;
94 }

```

### 第三题： [蓝桥杯 2023 省 Java A 与或异或](#)

这题本来就是填空题，本质上是有10个空，每个空可以填三种运算，dfs跑之，复杂度 $O(3^n * 10)$ 。

写一些工具类的函数可以减少点工作量。

```

1  int op[11];
2  int res[11];
3  int cnt=0;
4
5  int ope(int a,int b,int x){
6      if(x==1) return (a&b);
7      if(x==2) return (a|b);
8      if(x==3) return (a^b);
9  }
10
11 void dfs(int dep){
12     if(dep==11){
13         res[1]=ope(1,0,op[1]);
14         res[2]=ope(0,1,op[2]);

```

```

15     res[3]=ope(1,0,op[3]);
16     res[4]=ope(0,1,op[4]);
17     res[5]=ope(res[1],res[2],op[5]);
18     res[6]=ope(res[2],res[3],op[6]);
19     res[7]=ope(res[3],res[4],op[7]);
20     res[8]=ope(res[5],res[6],op[8]);
21     res[9]=ope(res[6],res[7],op[9]);
22     res[10]=ope(res[8],res[9],op[10]);
23     if(res[10]==1) cnt++;
24     return;
25 }
26 op[dep]=1;
27 dfs(dep+1);
28 op[dep]=2;
29 dfs(dep+1);
30 op[dep]=3;
31 dfs(dep+1);
32 }
33
34 void solve(){
35     dfs(1);
36     cout<<cnt;
37 }

```

#### 第四题： [蓝桥杯 2024 国 Python B 不同的总分值](#)

填空题，每个数有选和不选两种状态，总共 $2^{10}$ 种，我们用`set`去重。

```

1  int a[11]={0,5,5,10,10,15,15,20,20,25,25};
2  set<int> ans;
3  int cur=0;
4
5  void dfs(int dep){
6      if(dep==11){
7          ans.insert(cur);
8          return;
9      }
10     dfs(dep+1);
11     cur+=a[dep];
12     dfs(dep+1);
13     cur-=a[dep];
14 }
15
16 void solve(){
17     dfs(1);
18     cout<<ans.size();
19 }

```

## 第五题：蓝桥杯 2023 省 Python A 分糖果 - 洛谷

不考虑糖果够不够的话，每个孩子有18种选法， $18^7 = 6e8$ ，另外可以用糖果数量小于0的情况直接结束剪枝。

```
1  int a=9;
2  int b=16;
3  ll cnt=0;
4
5  void dfs(int dep){
6      if(a<0 or b<0) return;
7      if(dep==8){
8          if(a==0 and b==0) cnt++;
9          return;
10     }
11     fp(i,0,5) fp(j,0,5){
12         if(i+j<2 or i+j>5) continue;
13         a-=i;
14         b-=j;
15         dfs(dep+1);
16         a+=i;
17         b+=j;
18     }
19 }
20
21 void solve(){
22     // dfs(1);
23     // cout<<cnt;
24     cout<<5067671;
25 }
```

## 第六题：Team Building

每个人有选和不选两种状态，共 $2^n$ 种选法，每种选法要遍历 $m$ 对不兼容对，复杂度 $O(2^n * m)$ 。

```
1  const int N = 19;
2  const int M = 200;
3
4  ll n,m,k;
5  vector<ll> a(N);
6  vector<bool> vis(N);
7  vector<array<ll,3>> p(M);
8  ll ans=-INF;
9  ll cur=0;
10
11 void dfs(ll dep,ll cnt){
12     if(dep==n+1){
13         if(cnt!=k) return;
14         ll tmp=0;
15         fp(i,1,m) if(vis[p[i][0]] && vis[p[i][1]]) tmp+=p[i][2];
16         ans=max(ans,cur-tmp);
17         return;
18     }
```

```

18     }
19     dfs(dep+1,cnt);
20     if(cnt==k) return;
21     cur+=a[dep];
22     vis[dep]=1;
23     dfs(dep+1,cnt+1);
24     vis[dep]=0;
25     cur-=a[dep];
26 }
27
28 void solve(){
29     cin>>n>>m>>k;
30     fp(i,1,n) cin>>a[i];
31     fp(i,1,m) cin>>p[i][0]>>p[i][1]>>p[i][2];
32     dfs(1,0);
33     cout<<ans;
34 }

```

## 第七题: [E - Exhibition Booth Arrangement](#)

总共有  $n!$  种排序方法, 对于每一种, 我们可以  $O(n^2)$  检查这种排序方法能否在  $k$  次以内换到, 如果能,  $O(n)$  计算总值, 对于所有的总值取最大。复杂度  $O(n! * n^2)$ 。

```

1 void solve(){
2     ll n,k;
3     cin>>n>>k;
4     vector c(n+1,vector<ll>(n+1));
5     for(int i=1;i<=n;++i){
6         for(int j=1;j<=n;++j){
7             cin>>c[i][j];
8         }
9     }
10    vector<bool> vis(n+1);
11    vector<ll> a(n+1);
12    ll ans=0;
13
14    auto check=[&]() ->bool {
15        vector<ll> b(n+1);
16        ll cnt=0;
17        for(int i=1;i<=n;++i) b[i]=a[i];
18        for(int i=1;i<=n;++i)
19            if(b[i]!=i){
20                cnt++;
21                for(int j=i+1;j<=n;++j)
22                    if(b[j]==i) swap(b[i],b[j]);
23            }
24        if(cnt<=k) return 1;
25        return 0;
26    };
27
28    auto cal=[&]() ->ll {
29        ll res=0;
30        for(int i=1;i<=n-1;++i) res+=c[a[i]][a[i+1]];

```

```

31         res+=c[a[n]][a[1]];
32         return res;
33     };
34
35     auto dfs=[&](auto&& self, ll dep) {
36         if(dep==n+1){
37             if(check()){
38                 ans=max(ans,cal());
39             }
40             return;
41         }
42         for(int i=1;i<=n;++i){
43             if(vis[i]) continue;
44             vis[i]=1;
45             a[dep]=i;
46             self(self,dep+1);
47             vis[i]=0;
48         }
49     };
50
51     dfs(dfs,1);
52     cout<<ans;
53 }

```

## 第八题: [E - Assortment of Sweets](#)

这题很有意思。

没看数据范围之前一眼背包板题，结果一看数据范围，值域 $10^{12}$ ，直接吓哭了。

好消息是 $n$ 只有40，看起来能搜索。坏消息是 $2^{40} = 10^{12}$ ，感觉怎么剪枝都会T。

你回想前面做的这么多的爆搜题，总觉得正确的复杂度应该是 $2^{20}$ 才对，即 $2^{\frac{n}{2}}$ 。

$\frac{n}{2}$  是什么东西呢，是跑一半的物品吗？那么另一半怎么办，也跑一遍吗？想到这里大概就有思路了。

把 $n$ 个物品拆成两半，分别跑 $dfs$ ，然后把两边的信息组合起来。

```

1  vector<ll> a(41);
2  ll n,s;
3  ll now=0;
4  ll cnt=0;
5  ll u;
6  ll v;
7  map<ll,ll> cnt1;
8  map<ll,ll> cnt2;
9
10 void dfs1(ll dep){
11     if(dep==u+1){
12         cnt1[now]++;
13         return;
14     }
15     now+=a[dep];
16     dfs1(dep+1);
17     now-=a[dep];

```



```

18     dfs1(dep+1);
19 }
20
21 void dfs2(ll dep){
22     if(dep==n+1){
23         cnt2[now]++;
24         return;
25     }
26     now+=a[dep];
27     dfs2(dep+1);
28     now-=a[dep];
29     dfs2(dep+1);
30 }
31
32 void solve(){
33     cin>>n>>s;
34     u=n/2;
35     v=n-u;
36     for(int i=1;i<=n;++i){
37         cin>>a[i];
38     }
39     now=0;
40     dfs1(1);
41     now=0;
42     dfs2(u+1);
43     ll ans=0;
44     for(auto &[x,y]:cnt1){
45         ans+=cnt2[s-x]*y;
46     }
47     cout<<ans;
48 }

```

## 第九题: [E - Cargo Delivery Truck](#)

依旧考虑搜索。

理论上搜索能达到  $15^{15} = 4e17$ , 感觉也是怎么都会T。

但是, 这题确实能是剪枝过去的, 剪枝很神秘吧 (

因为只要有一种方案成功了, 我们就可以判断可行, 然后不用跑了, 所以我们尽量让更可能成功的方案早点出现。

为了做到这一点, 我们要先装承载量大的卡车, 即按承载量大降序排序卡车。

或者如果前几个包裹就已经无论如何都装不了了, 那么我们就可以判断绝对没有可行方案, 后面也不用跑了。

为了做到这一点, 我们要先装重的包裹, 即按重量降序排序包裹。

另外, 我们还可以提前判断两种不可能有可行方案的情况, 即最重的包裹重于承载量最大的卡车, 和包裹的总重量大于卡车的总承载量。

在这些神秘剪枝的帮助下, 我们可以顺利通过此题。

```

1 vector<ll> w(16);
2 vector<ll> c(16);

```

```

3  ll n,m;
4
5  bool dfs(ll dep){
6      if(dep==n+1){
7          return 1;
8      }
9      for(int i=1;i<=m;++i){
10         if(c[i]>=w[dep]){
11             c[i]-=w[dep];
12             bool flag=dfs(dep+1);
13             if(flag) {return 1;}
14             c[i]+=w[dep];
15         }
16     }
17     return 0;
18 }
19
20 void solve(){
21     cin>>n>>m;
22     ll sw=0;
23     ll sc=0;
24     ll maxw=0;
25     ll maxc=0;
26     for(int i=1;i<=n;++i) {cin>>w[i];sw+=w[i];maxw=max(maxw,w[i]);}
27     sort(w.begin()+1,w.begin()+n,greater<ll>());
28     for(int j=1;j<=m;++j) {cin>>c[j];sc+=c[j];maxc=max(maxc,c[j]);}
29     sort(c.begin()+1,c.begin()+m,greater<ll>());
30     if(sw>sc or maxw>maxc){
31         cout<<"No";
32         return;
33     }
34
35     bool flag=dfs(1);
36     if(flag) cout<<"Yes";
37     else cout<<"No";
38 }

```

## 第十题: [E - Count the Types of Flowers](#)

好玩而又暴力的题目。

如果我们已经知道了区间 $[l,r]$ 中 不同类型的花卉的个数以及每种花卉的数量 的话, 我们是可以 $O(1)$ 得到 $[l+1,r]$ ,  $[l-1,r]$ ,  $[l,r-1]$ 和 $[l,r+1]$ 这四个区间之一的 不同类型的花卉的个数以及每种花卉的数量 的。

所以, 对于每个查询, 我们都可以通过移动前一个查询的左右端点 (即  $l$  和  $r$ ) , 来得到下一个查询的答案。

然而, 这样子的最坏情况, 每次我们都要移动 $O(n)$ 次端点, 才能到达下一个查询的区间。

为了减少端点的移动次数, 我们可以考虑把查询换一下顺序, 即先把所有的查询存下来, 对这些查询按照我们希望的顺序排序以减少端点的移动次数。完成所有的查询后再按照题目要求的顺序输出。

那么怎么样才可以减少移动次数呢, 我们考虑分块, 即将 $n$ 分为 $\sqrt{n}$ 个块, 每个块长度为 $\sqrt{n}$ 。

然后我们按照这样的方法对查询 $[l,r]$ 排序:

1. 先比较左端点  $l$  所在的块, 让左端点  $l$  所在的块更靠左的排在前面。

2. 如果左端点所在的块一样，那么如果左端点在偶数块，就按照  $r$  递增地排序，如果左端点在奇数块，就按照  $r$  递减地排序。

我们现在证明，按照这个排序方法，端点移动次数不超过  $O(n\sqrt{n} + q\sqrt{n})$ 。

对于左端点  $l$ ，它有两种移动，即块内移动和跨块移动。

对于块内移动，每次最多移动  $\sqrt{n}$ ，最多移动  $q$  次，即最多  $q\sqrt{n}$ 。

对于跨块移动，每次最多移动  $2\sqrt{n}$ ，最多跨  $\sqrt{n}$  次块，即最多  $O(n)$ 。

对于右端点  $r$ ，它对于每一个左端点所在的分块，都最多移动  $n$  次（即从最左移到最右或最右移到最左），而左端点最多只在  $\sqrt{n}$  个块中，所以  $r$  最多移动  $n\sqrt{n}$  次。

```
1  struct qry{
2      ll l,r,block,i;
3  };
4
5  bool cmp(qry a,qry b){
6      if(a.block!=b.block){
7          return a.block<b.block;
8      }
9      else if(a.block%2==0) return a.r<b.r;
10     else return a.r>b.r;
11 }
12
13 void solve(){
14     ll n,q;
15     cin>>n>>q;
16     vector<ll> p(n);
17     fp(i,0,n-1) cin>>p[i];
18     vector<qry> qrys;
19     ll d=(ll)sqrt(n);
20     fp(i,1,q){
21         ll l,r;
22         cin>>l>>r;
23         qrys.push_back({l-1,r-1,(l-1)/d,i});
24     }
25     sort(qrys.begin(),qrys.end(),cmp);
26
27     vector<ll> ans(q+1);
28     vector<ll> cnt(n+1);
29
30     ll cur1=0;
31     ll curr=0;
32     ll curn=1;
33     cnt[p[0]]++;
34
35     auto add=[&](ll x){
36         if(cnt[p[x]]==0) curn++;
37         cnt[p[x]]++;
38     };
39
40     auto remove=[&](ll x){
41         if(cnt[p[x]]==1) curn--;
42         cnt[p[x]]--;
43     };
```

```

44
45     fp(i,0,q-1){
46         auto tar=qrys[i];
47         ll l=tar.l;
48         ll r=tar.r;
49         ll id=tar.i;
50         while(curl>l){
51             add(curl-1);
52             curl--;
53         }
54         while(curl<l){
55             remove(curl);
56             curl++;
57         }
58         while(curr<r){
59             add(curr+1);
60             curr++;
61         }
62         while(curr>r){
63             remove(curr);
64             curr--;
65         }
66         ans[id]=curn;
67     }
68
69     fp(i,1,q){
70         cout<<ans[i]<<"\n";
71     }
72 }

```

## 第十一题: [F - Authentic Traveling Salesman Problem](#)

和第十题一样的分块思路。

```

1  struct pos{
2      ll x,y,bx,id;
3  };
4
5  bool cmp(pos a,pos b){
6      if(a.bx!=b.bx){
7          return a.bx<b.bx;
8      }
9      if(a.bx%2==0) return a.y<b.y;
10     else return a.y>b.y;
11 }
12
13 void solve(){
14     ll n;
15     cin>>n;
16     vector<pos> p(n-1);
17     ll x,y;
18     cin>>x>>y;
19     if(n==1){

```

```
20         cout<<1;
21         return;
22     }
23     ll d=(ll)sqrt((ll)(4e14)/n);
24     fp(i,0,n-2){
25         cin>>x>>y;
26         p[i]={x-1,y-1,(x-1)/d,i+2};
27     }
28     cout<<1<<" ";
29     sort(p.begin(),p.end(),cmp);
30     fp(i,0,n-2){
31         cout<<p[i].id<<" ";
32     }
33 }
```